

The Predeclared Types

Go has a set of built-in types similar to other languages (booleans, integers, floats, and strings).

The Zero Value

- Go assigns a default "zero value" to any variable that is declared but not assigned a value.
- This removes sources of bugs found in uninitialized variables in languages like C.
 - Integers/Floats: 0
 - Booleans: false
 - Strings: "" (empty string)

Literals

Literals are explicitly specified values in source code. They are untyped by default, meaning their type is inferred from context.

- Integer Literals:**
 - Base 10 is default.
 - Prefixes: 0b (binary), 0o (octal), 0x (hexadecimal).
 - Underscores: Can be inserted into numeric literals for readability (e.g., 1_234). They have no effect on the value.
- Floating-point Literals:** Use decimal points or the letter e for exponents. Can also use hex format (0x...p...).
- Rune Literals:**
 - Represent characters and are surrounded by single quotes ' (e.g., 'a').
 - Double quotes and single quotes are not interchangeable in Go.
- String Literals:**
 - Interpreted: Surrounded by double quotes ". Interprets escape characters (like \n).
 - Raw: Surrounded by backticks `. Can contain any character except a backtick, including newlines. Useful for multiline strings or regular expressions.

Numeric Types

Go has 12 distinct numeric types grouped into integers, floating-point, and complex numbers

Integers

- Specific types:** int8, int16, int32, int64 and their unsigned counterparts uint8 through uint64.

Type name	Value range
int8	−128 to 127
int16	−32768 to 32767
int32	−2147483648 to 2147483647
int64	−9223372036854775808 to 9223372036854775807
uint8	0 to 255
uint16	0 to 65535
uint32	0 to 4294967295
uint64	0 to 18446744073709551615

- Special aliases:**
 - byte: Alias for uint8.
 - int: Platform-dependent (32-bit on 32-bit systems, 64-bit on 64-bit systems).
Note: int is a distinct type; you cannot assign an int32 to an int without casting.
 - uint: Unsigned platform-dependent integer.
- Usage Rule:** Use int unless you need a specific size for a file format or network protocol.

Floating-point

- Types:** float32 and float64.
- Rule:** Use float64 by default (it is the default for literals). float32 has limited precision.
- Comparison:** Do not use == to compare floats due to inexactness. Use a "delta" or "epsilon" variance check.

Complex Numbers

- Types:** complex64 (uses float32 components) and complex128 (uses float64 components).
- Functions:** real(), imag(), and complex().
- Note:** These are rarely used outside of specific mathematical domains.

Strings and Runes

- Strings:** Immutable sequences of values. You can reassign a string variable, but you cannot modify the existing string's content.
- Runes:** The type rune is an alias for int32. It represents a single Unicode code point.
- Idiom:** Use rune when referring to a character and int32 when referring to numbers to clarify intent.

Type Discipline

Go is strict about types.

- No Automatic Type Promotion:** You cannot add a float64 to an int or even an int to an int32 without explicit conversion.
- Explicit Conversion:** You must cast types manually (e.g., float64(x) + y).
- No "Truthiness":** Go does not treat non-zero numbers or non-empty strings as true. You cannot cast pointers or integers to bool. You must use comparison operators (e.g., x == 0).

Variable Declaration

Go offers multiple ways to declare variables to communicate intent.

var vs :=

- var x int = 10:** The most verbose style.
- var x = 10:** Infers the type from the value (default for integer literals is int).
- var x int:** Declares a variable with its zero value. Idiomatic use.
- x := 10 (Short Declaration):** Declares and assigns. Only legal inside functions.
 - Can assign to existing variables as long as at least one variable on the left side is new.

Guidelines for Use:

- Use := inside functions for most cases.
- Use var x int (without =) when you want to initialize a variable to zero.
- Use var x byte = 20 when you need a specific type that differs from the literal's default type.
- Avoid package-level variables; they make data flow hard to track.

Constants (const)

- Declared with const. Used to name literals.
- Limit: Constants can only hold values computable at compile-time (numbers, strings, booleans, runes).
- Typed vs. Untyped:**
 - Untyped (const x = 10): Flexible. Acts like a literal and can be used with any compatible type (e.g., can be assigned to int, float64, or byte).
 - Typed (const x int = 10): Strict. Can only be assigned to variables of that specific type.

Unused Variables

- Rule:** It is a compile-time error to declare a local variable and not read its value.
- This rule helps clean up code and prevent bugs.
- Exception:** Unused constants are allowed (they are simply stripped out at compile time).

Naming Conventions

- Case Sensitivity:** a and A are different variables.
- CamelCase:** Go uses CamelCase (e.g., indexCounter), not snake_case (index_counter).
- Visibility:** The case of the first letter determines package visibility (Uppercase = public/exported, Lowercase = private/unexported).
- Length:** Keep names short (i, k, v) for small scopes (like loops). Use descriptive names for larger scopes (package-level).
- Naming Trick:** Go allows any Unicode character considered a letter or digit, but using look-alike characters (e.g., Greek omicron vs. 'o') is valid but dangerous code.